Document number: P3345R0
Date: 2024-06-28
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2020
Reply to: Jens Maurer
jens.maurer@gmx.net

# Core Language Working Group "ready" Issues for the June, 2024 meeting

References in this document reflect the section and paragraph numbering of document WG21 N4981.

---

### 233. References vs pointers in UDC overload resolution

**Section:** 9.4.4 [dcl.init.ref] **Status:** ready **Submitter:** Matthias Meixner **Date:** 9 Jun 2000

There is an inconsistency in the handling of references vs pointers in user defined conversions and overloading. The reason for that is that the combination of 9.4.4 [dcl.init.ref] and 7.3.6 [conv.qual] circumvents the standard way of ranking conversion functions, which was probably not the intention of the designers of the standard.

Let's start with some examples, to show what it is about:

```
struct Z { Z(){} };

struct A {
   Z x;

   operator Z *() { return &x; }
   operator const Z *() { return &x; }
};

struct B {
   Z x;

   operator Z &() { return x; }
   operator const Z &() { return x; }
};

int main()
{
   A a;
   Z *a1=a;
   const Z *a2=a; // not ambiguous

   B b;
   Z &b1=b;
   const Z &b2=b; // ambiguous
}
```

So while both classes `A` and `B` are structurally equivalent, there is a difference in operator overloading. I want to start with the discussion of the pointer case (`const Z *a2=a;`): 12.2.4 [over.match.best] is used to select the best viable function. Rule 4 selects `A::operator const Z*()` as best viable function using 12.2.4.3 [over.ics.rank] since the implicit conversion sequence `const Z*` -> `const Z*` is a better conversion sequence than `Z*` -> `const Z*`.

So what is the difference to the reference case? Cv-qualification conversion is only applicable for pointers according to 7.3.6 [conv.qual]. According to 9.4.4 [dcl.init.ref] paragraphs 4-7 references are initialized by binding using the concept of reference-compatibility. The problem with this is, that in this context of binding, there is no conversion, and therefore there is also no comparing of conversion sequences. More exactly all conversions can be considered identity conversions according to 12.2.4.2.5 [over.ics.ref] paragraph 1, which compare equal and which has the same effect. So binding `const Z*` to `const Z*` is as good as binding `const Z*` to `Z*` in terms of overloading. Therefore `const Z &b2=b;` is ambiguous. [12.2.4.2.5 [over.ics.ref] paragraph 5 and 12.2.4.3 [over.ics.rank] paragraph 3 rule 3 (S1 and S2 are reference bindings ...) do not seem to apply to this case]

There are other ambiguities, that result in the special treatment of references: Example:

```
struct A {int a;};
struct B: public A { B() {}; int b;};

struct X {
   B x;
   operator A &() { return x; }
   operator B &() { return x; }
};

main()
{
   X x;
   A &g=x; // ambiguous
}
```

Since both references of class `A` and `B` are reference compatible with references of class `A` and since from the point of ranking of implicit conversion sequences they are both identity conversions, the initialization is ambiguous.

So why should this be a defect?

- References behave fundamentally different from pointers in combination with user defined conversions, although there is no reason to have this different treatment.
- This difference only shows up in combination with user defined conversion sequences, for all other cases, there are special rules, e.g. 12.2.4.3 [over.ics.rank] paragraph 3 rule 3.

So overall I think this was not the intention of the authors of the standard.

So how could this be fixed? For comparing conversion sequences (and only for comparing) reference binding should be treated as if it was a normal assignment/initialization and cv-qualification would have to be defined for references. This would affect 9.4.4 [dcl.init.ref] paragraph 6, 7.3.6 [conv.qual] and probably 12.2.4.3 [over.ics.rank] paragraph 3.

Another fix could be to add a special case in 12.2.4 [over.match.best] paragraph 1.

**CWG 2023-06-13**

It was noted that the second example is not ambiguous, because a derived-to-base conversion is compared against an identity conversion. However, 12.2.4.2.5 [over.ics.ref] paragraph 1 needs a wording fix so that it applies to conversion functions as well. CWG opined that the first example be made valid, by adding a missing tie-breaker for the conversion function case.

**Proposed resolution (approved by CWG 2024-04-19):**

1. Change in 12.2.4.1 [over.match.best.general] bullet 2.2 as follows:

    - ...
    - the context is an initialization by user-defined conversion (see 9.4 [dcl.init], 12.2.2.6 [over.match.conv], and 12.2.2.7 [over.match.ref]) and the standard conversion sequence

from the ~~return type~~ **result** of F1 to the destination type (i.e., the type of the entity being initialized) is a better conversion sequence than the standard conversion sequence from the ~~return type~~ **result** of F2 to the destination type

- ...

2. Add a new sub-bullet to 12.2.4.3 [over.ics.rank] bullet 3.2 as follows:

- ...
- S1 and S2 include reference bindings (9.4.4 [dcl.init.ref]), and the types to which the references refer are the same type except for top-level cv-qualifiers, and the type to which the reference initialized by S2 refers is more cv-qualified than the type to which the reference initialized by S1 refers~~.~~ [ Example: ... -- end example ] **or, if not that,**
- `S1` and `S2` bind the same reference type "reference to `T`" and have source types `V1` and `V2`, respectively, where the standard conversion sequence from `V1*` to `T*` is better than the standard conversion sequence from `V2*` to `T*`. [ Example:

```
struct Z {};

struct A {
  operator Z&();
  operator const Z&();       // #1
};

struct B {
  operator Z();
  operator const Z&&();      // #2
};

const Z& r1 = A();           // OK, uses #1
const Z&& r2 = B();          // OK, uses #2
```

--- end example]

---

**2144. Function/variable declaration ambiguity**

**Section:** 9.5.1 [dcl.fct.def.general] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-06-19

The following fragment,

```
int f() {};
```

is syntactically ambiguous. It could be either a *function-definition* followed by an *empty-declaration*, or it could be a *simple-declaration* whose *init-declarator* has the *brace-or-equal-initializer* `{}`. The same is true of a variable declaration

```
int a {};
```

since *function-definition* simply uses the term *declarator* in its production.

**Additional notes (May, 2024)**

Issue 2876 introduced a framework to distinguish the parsing. Its resolution was extended to also resolve this issue.

---

**2561. Conversion to function pointer for lambda with explicit object parameter**

**Section:** 7.5.5.2 [expr.prim.lambda.closure] **Status:** ready **Submitter:** Barry Revzin **Date:** 2022-02-14

P0847R7 (Deducing this) (approved October, 2021) added explicit-object member functions. Consider:

```
struct C {
  C(auto) { }
};

void foo() {
  auto l = [](this C) { return 1; };
  void (*fp)(C) = l;
  fp(1); // same effect as decltype(l){}() or decltype(l){}(1) ?
}
```

Subclause 7.5.5.2 [expr.prim.lambda.closure] paragraph 8 does not address explicit object member functions:

> The closure type for a non-generic *lambda-expression* with no *lambda-capture* whose constraints (if any) are satisfied has a conversion function to pointer to function with C++ language linkage (9.11 [dcl.link]) having the same parameter and return types as the closure type's function call operator. The conversion is to "pointer to noexcept function" if the function call operator has a non-throwing exception specification. The value returned by this conversion function is the address of a function F that, when invoked, has the same effect as invoking the closure type's function call operator on a default-constructed instance of the closure type. F is a constexpr function if...

Suggested resolution [SUPERSEDED]:

1. Change in 7.5.5.2 [expr.prim.lambda.closure] paragraph 8 as follows:

> ... The value returned by this conversion function is
> - **for a *lambda-expression* whose *parameter-declaration-clause* has an explicit object parameter, the address of the function call operator (7.6.2.2 [expr.unary.op];**
> - **otherwise,** the address of a function F that, when invoked, has the same effect as invoking the closure type's function call operator on a default-constructed instance of the closure type.
>
> F is a constexpr function if... is an immediate function.
>
> **[ *Example*:**
>
> ```
> struct C {
>   C(auto) { }
> };
>
> void foo() {
>   auto a = [](C) { return 0; };
>   int (*fp)(C) = a;   // OK
>   fp(1);              // same effect as decltype(a){}(1)
>   auto b = [](this C) { return 1; };
>   fp = b;             // OK
>   fp(1);              // same effect as (&decltype(b)::operator())(1)
> }
> ```
>
> ***-- end example*** **]**

2. Change in 7.5.5.2 [expr.prim.lambda.closure] paragraph 11 as follows:

> The value returned by any given specialization of this conversion function template is
> - **for a *lambda-expression* whose *parameter-declaration-clause* has an explicit object parameter, the address of the corresponding function call operator template**

**specialization (7.6.2.2 [expr.unary.op]);**

- **otherwise,** the address of a function F that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization on a default-constructed instance of the closure type.

F is a constexpr function if...

**CWG 2023-06-17**

Requesting guidance from EWG with paper issue 1689.

**Additional notes (October, 2023)**

Additional examples demonstrating implementation divergence between clang and MSVC:

```
struct Any { Any(auto) {} };
auto x = [](this auto self, int x) { return x; };
auto y = [](this Any self, int x) { return x; };
auto z = [](this int (*self)(int), int x) { return x; };
int main() {
  x(1);
  y(1);
  z(1);
  int (*px)(int) = +x; // MSVC
  int (*py1)(int) = +y; // MSVC
  int (*py2)(Any, int) = +y; // Clang
  int (*pz1)(int) = +z; // MSVC
  int (*pz2)(int (*)(int), int) = +z; // Clang
}
```

**Additional notes (November, 2023)**

Additional example:

```
auto c2 = [](this auto self) { return sizeof(self); };
struct Derived2 : decltype(c) { int value; } d2;
struct Derived3 : decltype(c) { int value[10]; } d3;
```

For MSVC, `d2() == 4` and `d3() == 40`, but `+d2` and `+d3` both point to functions returning `1`.

**EWG 2023-11-07**

Move forward with option 1 "punt" from D3031 for C++26. A future paper can explore other solutions.

**Proposed resolution (approved by CWG 2024-04-19):**

1. Change the example in 7.5.5.1 [expr.prim.lambda.general] paragraph 6 as follows:

```
~~int i = [](int i, auto a) { return i; }(3, 4);  // OK, a generic lambda~~
~~int j = []<class T>(T t, int i) { return i; }(3, 4);  // OK, a generic lambda~~
auto x = [](int i, auto a) { return i; };              // OK, a generic lambda
auto y = [](this auto self, int i) { return i; };      // OK, a generic lambda
auto z = []<class T>(int i) { return i; };             // OK, a generic lambda
```

2. Change in 7.5.5.2 [expr.prim.lambda.closure] paragraph 9 as follows:

> The closure type for a non-generic *lambda-expression* with no *lambda-capture* **and no explicit object parameter (9.3.4.6 [dcl.fct])** whose constraints (if any) are satisfied has a conversion

function to pointer to function with C++ language linkage (9.11 [dcl.link]) having the same parameter and return types as the closure type's function call operator. ...

3. Change in 7.5.5.2 [expr.prim.lambda.closure] paragraph 10 as follows:

> For a generic lambda with no *lambda-capture* **and no explicit object parameter (9.3.4.6 [dcl.fct])**, the closure type has a conversion function template to pointer to function. ...

**CWG 2023-11-09**

Keeping in review status in anticipation of a paper proposing reasonable semantics for the function pointer conversions.

**EWG 2024-03-18**

Progress with option #1 of P3031R0, affirming the direction of the proposed resolution.

---

### 2588. friend declarations and module linkage

**Section:** 11.8.4 [class.friend] **Status:** ready **Submitter:** Nathan Sidwell **Date:** 2022-05-26

Consider:

```
export module Foo;
class X {
  friend void f(X); // #1 linkage?
};
```

Subclause 11.8.4 [class.friend] paragraph 4 gives #1 external linkage:

> A function first declared in a friend declaration has the linkage of the namespace of which it is a member (6.6 [basic.link]).

(There is no similar provision for friend classes first declared in a class.)

However, 6.6 [basic.link] bullet 4.8 gives it module linkage:

> ... otherwise, if the declaration of the name is attached to a named module (10.1 [module.unit]) and is not exported (10.2 [module.interface]), the name has module linkage;

Subclause 10.2 [module.interface] paragraph 2 does not apply:

> A declaration is *exported* if it is declared within an *export-declaration* and inhabits a namespace scope or it is
>
> - a *namespace-definition* that contains an exported declaration, or
> - a declaration within a header unit (10.3 [module.import]) that introduces at least one name.

Also consider this related example:

```
export module Foo;
export class Y;
// maybe many lines later, or even a different partition of Foo
class Y {
```

```
  friend void f(Y); // #2 linkage?
};
```

- Should the friend's linkage be affected by the linkage of the befriending class? In this example, #2 would therefore have external linkage, as `Y` is exported.
- Or should the friend's linkage be affected by the presence or absence of `export` on the class definition itself? In this example, #2 would thus have module linkage.
- Or should the friend's linkage be determined ignoring any enclosing `export` and ignoring whether the enclosing class is exported, per 11.8.4 [class.friend] paragraph 4 (alone)?
- Or should the friend's linkage be as-if the declaration inhabited its nearest enclosing namespace scope, without the `friend`?

See issue 2607 for a similar question about enumerators.

**Additional note (May, 2022):**

Forwarded to EWG with paper issue 1253, by decision of the CWG chair.

**EWG telecon 2022-06-09**

Consensus: "A friend's linkage should be affected by the presence/absence of export on the containing class definition itself, but ONLY if the friend is a definition", pending confirmation by electronic polling.

**Proposed resolution (June, 2022) (approved by CWG 2023-01-27) [SUPERSEDED]:**

1. Change in 6.6 [basic.link] paragraph 4 as follows:

> ... The name of an entity that belongs to a namespace scope that has not been given internal linkage above and that is the name of
> - a variable; or
> - a function; or
> - a named class (11.1 [class.pre]), or an unnamed class defined in a typedef declaration in which the class has the typedef name for linkage purposes (9.2.4 [dcl.typedef]); or
> - a named enumeration (9.7.1 [dcl.enum]), or an unnamed enumeration defined in a typedef declaration in which the enumeration has the typedef name for linkage purposes (9.2.4 [dcl.typedef]); or
> - an unnamed enumeration that has an enumerator as a name for linkage purposes (9.7.1 [dcl.enum]); or
> - a template
>
> has its linkage determined as follows:
> - **if the entity is a function or function template first declared in a friend declaration and that declaration is a definition, the name has the same linkage, if any, as the name of the enclosing class (11.8.4 [class.friend]);**
> - **otherwise, if the entity is a function or function template declared in a friend declaration and a corresponding non-friend declaration is reachable, the name has the linkage determined from that prior declaration,**
> - **otherwise,** if the enclosing namespace has internal linkage, the name has internal linkage;
> - otherwise, if the declaration of the name is attached to a named module (10.1 [module.unit]) and is not exported (10.2 [module.interface]), the name has module linkage;
> - otherwise, the name has external linkage.

2. Remove 11.8.4 [class.friend] paragraph 4:

> ~~A function first declared in a friend declaration has the linkage of the namespace of which it is a member (6.6 [basic.link]). Otherwise, the function retains its previous linkage (9.2.2 [dcl.ste]).~~

**EWG electronic poll 2022-06**

Consensus for "A friend's linkage should be affected by the presence/absence of export on the containing class definition itself, but ONLY if the friend is a definition (option #2, modified by Jason's suggestion). This resolves CWG2588." See vote.

**Proposed resolution (approved by CWG 2024-03-20):**

1. Change in 6.6 [basic.link] paragraph 4 as follows:

> ... The name of an entity that belongs to a namespace scope that has not been given internal linkage above and that is the name of
> - a variable; or
> - a function; or
> - a named class (11.1 [class.pre]), or an unnamed class defined in a typedef declaration in which the class has the typedef name for linkage purposes (9.2.4 [dcl.typedef]); or
> - a named enumeration (9.7.1 [dcl.enum]), or an unnamed enumeration defined in a typedef declaration in which the enumeration has the typedef name for linkage purposes (9.2.4 [dcl.typedef]); or
> - an unnamed enumeration that has an enumerator as a name for linkage purposes (9.7.1 [dcl.enum]); or
> - a template
>
> has its linkage determined as follows:
> - **if the entity is a function or function template first declared in a friend declaration and that declaration is a definition and the enclosing class is defined within an *export-declaration*, the name has the same linkage, if any, as the name of the enclosing class (11.8.4 [class.friend]);**
> - **otherwise, if the entity is a function or function template declared in a friend declaration and a corresponding non-friend declaration is reachable, the name has the linkage determined from that prior declaration,**
> - **otherwise,** if the enclosing namespace has internal linkage, the name has internal linkage;
> - otherwise, if the declaration of the name is attached to a named module (10.1 [module.unit]) and is not exported (10.2 [module.interface]), the name has module linkage;
> - otherwise, the name has external linkage.

2. Remove 11.8.4 [class.friend] paragraph 4:

> ~~A function first declared in a friend declaration has the linkage of the namespace of which it is a member (6.6 [basic.link]). Otherwise, the function retains its previous linkage (9.2.2 [dcl.stc]).~~

---

### 2728. Evaluation of conversions in a *delete-expression*

**Section:** 7.6.2.9 [expr.delete] **Status:** ready **Submitter:** Jiang An **Date:** 2023-05-05

Subclause 7.6.2.9 [expr.delete] paragraph 4 specifies:

> The *cast-expression* in a *delete-expression* shall be evaluated exactly once.

Due to the reference to the syntactic non-terminal *cast-expression*, it is unclear whether that includes the conversion to pointer type specified in 7.6.2.9 [expr.delete] paragraph 2.

**Proposed resolution (approved by CWG 2024-03-01) [SUPERSEDED]:**

1. Change in 7.6.2.9 [expr.delete] paragraph 1 and paragraph 2 as follows:

... The operand shall be **of class type or a prvalue** of pointer to object type ~~or of class type~~. If of class type, the operand is contextually implicitly converted (7.3 [conv]) to a **prvalue** pointer to object type. [ Footnote: ... ] **The converted operand is used in place of the original operand for the remainder of this subclause.** The *delete-expression* has type void.

~~If the operand has a class type, the operand is converted to a pointer type by calling the above-mentioned conversion function, and the converted operand is used in place of the original operand for the remainder of this subclause.~~ ...

2. Change in 7.6.2.9 [expr.delete] paragraph 4 as follows:

The ~~*cast-expression* in~~ **operand of** a *delete-expression* shall be evaluated exactly once.

**Proposed resolution (approved by CWG 2024-03-20):**

1. Change in 7.6.2.9 [expr.delete] paragraph 1 and paragraph 2 as follows:

... The first alternative is a single-object delete expression, and the second is an array delete expression. Whenever the delete keyword is immediately followed by empty square brackets, it shall be interpreted as the second alternative. [ Footnote: ... ] If the operand is of class type, it is contextually implicitly converted (7.3 [conv]) to a pointer to object type **and the converted operand is used in place of the original operand for the remainder of this subclause**. ~~[ Footnote: ...]~~ Otherwise, it shall be a prvalue of pointer to object type. The *delete-expression* has type void.

~~If the operand has a class type, the operand is converted to a pointer type by calling the above-mentioned conversion function, and the converted operand is used in place of the original operand for the remainder of this subclause.~~ ...

2. Delete 7.6.2.9 [expr.delete] paragraph 4:

~~The *cast-expression* in a *delete-expression* shall be evaluated exactly once.~~

---

**2818. Use of predefined reserved identifiers**

**Section:** 5.10 [lex.name] **Status:** ready **Submitter:** Jiang An **Date:** 2023-01-18

Subclause 5.10 [lex.name] paragraph 3 specifies:

> In addition, some identifiers appearing as a *token* or *preprocessing-token* are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required.
>
> - Each identifier that contains a double underscore __ or begins with an underscore followed by an uppercase letter is reserved to the implementation for any use.
> - Each identifier that begins with an underscore is reserved to the implementation for use as a name in the global namespace.

That implies that uses of standard-specified predefined macros (15.11 [cpp.predefined]) or feature-test macros (17.3.2 [version.syn]) make the program ill-formed. This does not appear to be the intent.

**Proposed resolution (approved by CWG 2024-01-19) [SUPERSEDED]:**

Change in 5.10 [lex.name] paragraph 3 and add bullets as follows:

In addition, some identifiers appearing as a *token* or *preprocessing-token* are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required.

- Each identifier that contains a double underscore __ or begins with an underscore followed by an uppercase letter is reserved to the implementation for any use**, except for**
  - the macros specified in 15.11 [cpp.predefined],
  - the function-local predefined variable (9.5.1 [dcl.fct.def.general]), and
  - the macros and identifiers declared in the standard library as specified in Clause 17 [support] through Clause 33 [thread], and Clause Annex D [depr]. [ Note: This affects macros in 17.3.2 [version.syn], 31.13.1 [cstdio.syn], and D.11 [depr.c.macros] as well as identifiers in 17.2.2 [cstdlib.syn]. -- end note ]
- Each identifier that begins with an underscore is reserved to the implementation for use as a name in the global namespace.

**CWG 2024-03-20**

The exceptions should not be specified using a specific list, which might become stale in the future.

**Proposed resolution (approved by CWG 2024-03-20):**

Change in 5.10 [lex.name] paragraph 3 and add bullets as follows:

In addition, some identifiers appearing as a *token* or *preprocessing-token* are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required.

- Each identifier that contains a double underscore __ or begins with an underscore followed by an uppercase letter**, other than those specified in this document (for example, `__cplusplus` (15.11 [cpp.predefined])),** is reserved to the implementation for any use.
- Each identifier that begins with an underscore is reserved to the implementation for use as a name in the global namespace.

---

**2819. Cast from null pointer value in a constant expression**

**Section:** 7.7 [expr.const] **Status:** ready **Submitter:** Jason Merrill **Date:** 2023-10-19

Subclause 7.7 [expr.const] bullet 5.14 was amended by P2738R1 to support certain casts from `void*` to object pointer types. The bullet specifies:

- ...
- a conversion from a prvalue P of type “pointer to cv void” to a pointer-to-object type T unless P points to an object whose type is similar to T;
- ...

This wording does not, but should, support null pointer values. The implementation burden is negligible.

**Proposed resolution (approved by CWG 2023-12-01):**

Change in 7.7 [expr.const] bullet 5.14 as follows:

- ...
- a conversion from a prvalue P of type “pointer to cv void” to a pointer-to-object type T unless P **is a null pointer value or** points to an object whose type is similar to T;
- ...

**CWG 2023-12-01**

CWG seeks approval from EWG for the design direction. See paper issue 1698.

**EWG 2024-03-18**

EWG approves.

**CWG 2024-04-19**

This issue is not a DR.

---

### 2836. Conversion rank of `long double` and extended floating-point types

**Section:** 6.8.6 [conv.rank] **Status:** ready **Submitter:** Joshua Cranmer **Date:** 2023-11-20

Consider:

```
auto f(long double x, std::float64_t y) {
  return x + y;
}
```

What is the return type of `f`?

Suppose an implementation uses the same size and semantics for all of `double`, `long double`, and `std::float64_t`. C23 prefers the IEEE interchange type (i.e. `std::float64_t`) over `long double` and `double`. In contrast, C++ chooses `long double`, which has a higher rank than `double`, but `std::float64_t` is specified to have the same rank as `double`.

This outcome conflicts with the documented goal of P1467 that C++ and C conversion rules be aligned.

Suggested resolution [SUPERSEDED]:

Change in 6.8.6 [conv.rank] bullet 2.5 as follows:

- ...
- An extended floating-point type with the same set of values as more than one cv-unqualified standard floating-point type has a rank equal to the **highest** rank ~~of double~~ **among such types**.

**Additional notes (December, 2023)**

The suggested resolution would make a conversion from `std::float64_t` to `double` not implicit, which is not desirable.

David Olsen, one of the authors of P1467, asserts that the deviation from C is intentional, and was discussed with the C floating-point study group.

Forwarding to EWG via paper issue 1699 to confirm that the deviation from C is intentional and thus an Annex C entry should be created.

**EWG 2024-03-18**

This issue should be closed as NAD.

Possible resolution [SUPERSEDED]:

Add a new paragraph in C.7.3 [diff.basic] as follows:

> **Affected subclause:** 6.8.6 [conv.rank]
> **Change:** Conversion rank of same-sized `long double` vs. `std::float64_t`
> **Rationale:** Provide implicit conversion from `std::float64_t` to `double`.
> **Effect on original feature:** Change of semantically well-defined feature.
> **Difficulty of converting:** Trivial: explicitly convert to the desired type.
> **How widely used:** Rarely.

**CWG 2024-04-19**

The name `std::float64_t` is not valid C. CWG resolved to adding a note to 6.8.6 [conv.rank] bullet 2.5 instead.

**Proposed resolution (approved by CWG 2024-06-26):**

Change in 6.8.6 [conv.rank] bullet 2.5 as follows:

- ...
- An extended floating-point type with the same set of values as more than one cv-unqualified standard floating-point type has a rank equal to the rank of double. **[ Note: The treatment of `std::float64_t` differs from that of the analoguous `_Float64` in C, for example on platforms where all of `long double`, `double`, and `std::float64_t` have the same set of values (see ISO/IEC 9899:2024 H.4.2). -- end note ]**

---

### 2858. Declarative *nested-name-specifier*s and *pack-index-specifier*s

**Section:** 7.5.4.3 [expr.prim.id.qual] **Status:** ready **Submitter:** Krystian Stasiowski **Date:** 2024-02-06

(From submission #499.)

Subclause 7.5.4.3 [expr.prim.id.qual] paragraph 2 specifies:

> ... A declarative *nested-name-specifier* shall not have a *decltype-specifier*. ...

This should have been adjusted by P2662R3 (Pack Indexing) to read *computed-type-specifier*.

**Proposed resolution (approved by CWG 2024-04-05):**

Change in 7.5.4.3 [expr.prim.id.qual] paragraph 2 as follows:

> ... A declarative *nested-name-specifier* shall not have a ~~*decltype-specifier*~~ ***computed-type-specifier***. ...

**CWG 2024-06-26**

This is not a DR.

---

### 2859. Value-initialization with multiple default constructors

**Section:** 9.4.1 [dcl.init.general] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-02-09

(From submission #501.)

Subclause 9.4.1 [dcl.init.general] paragraph 9 specifies (as modified by issue 2820):

> To *value-initialize* an object of type T means:
>
> - If T is a (possibly cv-qualified) class type (Clause 11 [class]), then
>   - if T has either no default constructor (11.4.5.2 [class.default.ctor]) or a default constructor that is user-provided or deleted, then the object is default-initialized;
>   - otherwise, the object is zero-initialized and then default-initialized.
> - ...

Per the specification, no zero-initialization occurs if the class has *any* user-provided default constructor, even if that constructor is not actually selected for the default initialization. This is observable in a constant expression, where reads of uninitialized data members are ill-formed.

**Proposed resolution (approved by CWG 2024-04-05):**

1. Change in 6.7.3 [basic.life] paragraph 1 as follows:

> The lifetime of an object or reference is a runtime property of the object or reference. A variable is said to have *vacuous initialization* if it is default-initialized**, no other initialization is performed,** and, if it is of class type or a (possibly multidimensional) array thereof, **a trivial constructor of** that class type ~~has a trivial default constructor~~ **is selected for the default-initialization**. The lifetime of an object of type T begins when: ...

2. Change in 9.4.1 [dcl.init.general] paragraph 9 as follows:

> To *value-initialize* an object of type T means:
> - If T is a (possibly cv-qualified) class type (Clause 11 [class]), then
>   - ~~if T has either no default constructor (11.4.5.2 [class.default.ctor]) or a default constructor that is user-provided or deleted, then the object is default-initialized;~~
>   - ~~otherwise, the object is zero-initialized and then default-initialized.~~

> **let `C` be the constructor selected to default-initialize the object, if any. If `C` is not user-provided, the object is first zero-initialized. In all cases, the object is then default-initialized.**
>
> - ...

---

### 2861. `dynamic_cast` on bad pointer value

**Section:** 7.6.1.7 [expr.dynamic.cast] **Status:** ready **Submitter:** Jim X **Date:** 2024-02-06

(From submission #497.)

Base-to-derived casts and cross-casts need to inspect the vtable of a polymorphic type. However, this is not defined as an "access" and there is no provision for undefined behavior analoguous to 7.2.1 [basic.lval] paragraph 11.

**Proposed resolution (approved by CWG 2024-06-26):**

1. Add a new paragraph after 7.6.1.7 [expr.dynamic.cast] paragraph 6 as follows:

If v is a null pointer value, the result is a null pointer value.

If v has type "pointer to *cv* U" and v does not point to an object whose type is similar (7.3.6 [conv.qual]) to U and that is within its lifetime or within its period of construction or destruction (11.9.5 [class.cdtor]), the behavior is undefined. If v is a glvalue of type U and v does not refer to an object whose type is similar to U and that is within its lifetime or within its period of construction or destruction, the behavior is undefined.

2. Change in 7.3.12 [conv.ptr] paragraph 3 as follows:

A prvalue **v** of type "pointer to cv D", where D is a complete class type, can be converted to a prvalue of type "pointer to cv B", where B is a base class (11.7 [class.derived]) of D. If B is an inaccessible (11.8 [class.access]) or ambiguous (6.5.2 [class.member.lookup]) base class of D, a program that necessitates this conversion is ill-formed. ~~The result of the conversion is a pointer to the base class subobject of the derived class object. The null pointer value is converted to the null pointer value of the destination type.~~ **If v is a null pointer value, the result is a null pointer value. Otherwise, if B is a virtual base class of D and v does not point to an object whose type is similar (7.3.6 [conv.qual]) to D and that is within its lifetime or within its period of construction or destruction (11.9.5 [class.cdtor]), the behavior is undefined. Otherwise, the result is a pointer to the base class subobject of the derived class object.**

---

**2864. Narrowing floating-point conversions**

**Section:** 9.4.5 [dcl.init.list] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-11-04

Consider:

```
float f = {1e100};
```

This is rejected as narrowing on all implementations. Issue 2723 made the example non-narrowing, which seems incorrect on an IEEE platform.

**Proposed resolution (approved by CWG 2024-04-19):**

Change in 9.4.5 [dcl.init.list] paragraph 7 as follows:

A *narrowing conversion* is an implicit conversion

- from a floating-point type to an integer type, or
- from a floating-point type T to another floating-point type whose floating-point conversion rank is neither greater than nor equal to that of T, except where the ~~source~~ **result of the conversion** is a constant expression and ~~the actual~~ **either its** value ~~after conversion~~ is **finite and** ~~within the range of values that can be represented (even if it cannot be represented exactly)~~ **the conversion did not overflow, or the values before and after the conversion are not finite**, or
- from an integer type ...

**Additional notes (April, 2024)**

According to the proposed wording, since NaNs are not finite, conversion of NaNs is always non-narrowing. However, the payload of the NaN might not be preserved when converting to a smaller floating-point type.

---

### 2865. Regression on result of conditional operator

**Section:** 7.6.16 [expr.cond] **Status:** ready **Submitter:** Christof Meerwald **Date:** 2024-01-14

Consider:

```
#include <concepts>

template <class T> T get();

template <class T>
using X = decltype(true ? get<T const&>() : get<T>());

struct C { };

static_assert(std::same_as<X<int>, int>);
static_assert(std::same_as<X<C>, C const>);  // #1
```

Before Issue 1895, #1 was well-formed. With the reformulation based on conversion sequences, #1 is now ill-formed because both conversion sequences can be formed.

**Proposed resolution (approved by CWG 2024-05-03):**

Change in 7.6.16 [expr.cond] bullet 4.3 as follows:

- ...
- If E2 is a prvalue or if neither of the conversion sequences above can be formed and at least one of the operands has (possibly cv-qualified) class type:
  - if T1 and T2 are the same class type (ignoring cv-qualification)**:**
    - ~~and~~ **if** T2 is at least as cv-qualified as T1, the target type is T2,
    - otherwise, no conversion sequence is formed for this operand;
  - otherwise, if T2 is a base class of T1, the target type is cv1 T2, where cv1 denotes the cv-qualifiers of T1~~,~~ **;**
  - otherwise, the target type is the type that E2 would have after applying the lvalue-to-rvalue (7.3.2 [conv.lval]), array-to-pointer (7.3.3 [conv.array]), and function-to-pointer (7.3.4 [conv.func]) standard conversions.

---

### 2867. Order of initialization for structured bindings

**Section:** 9.6 [dcl.struct.bind] **Status:** ready **Submitter:** Richard Smith **Date:** 2023-02-03

Consider:

```
auto [a, b] = f(X{});
```

If `X` is a tuple-like type, this is transformed to approximately the following:

```
auto e = f(X{});
T1 &a = get<0>(std::move(e));
T2 &b = get<1>(std::move(e));
```

However, the sequencing of the initializations of e, a, and b is not specified. Further, the temporary `X{}` should be destroyed after the initializations of a and b.

Possible resolution [SUPERSEDED]:

1. Change in 6.9.1 [intro.execution] paragraph 5 as follows:

> A *full-expression* is
> - an unevaluated operand (7.2.3 [expr.context]),
> - a *constant-expression* (7.7 [expr.const]),
> - an immediate invocation (7.7 [expr.const]),
> - an *init-declarator* (9.3 [dcl.decl]) or a *mem-initializer* (11.9.3 [class.base.init]), including the constituent expressions of the initializer,
> - the *initializer*s for all uniquely-named variables introduced by a structured binding declaration (9.6 [dcl.struct.bind]), including the constituent expressions of all initializers,
> - an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (6.7.7 [class.temporary]) whose lifetime has not been extended, or
> - an expression that is not a subexpression of another expression and that is not otherwise part of a full-expression.
>
> ...

2. Change in 9.6 [dcl.struct.bind] paragraph 4 as follows:

> ... Each $v_i$ is the name of an lvalue of type $T_i$ that refers to the object bound to $r_i$; the referenced type is $T_i$. **The initialization of e is sequenced before the initialization of any $r_i$. The initialization of $r_i$ is sequenced before the initialization of $r_j$ if i < j.**

**CWG 2024-05-03**

CWG tentatively agreed that all temporaries in a structured binding (including those from default arguments of `get` invocations) should persist until the semicolon on the declaration as written.

Possible resolution [SUPERSEDED]:

1. Change in 6.9.1 [intro.execution] paragraph 5 as follows:

> A *full-expression* is
> - an unevaluated operand (7.2.3 [expr.context]),
> - a *constant-expression* (7.7 [expr.const]),
> - an immediate invocation (7.7 [expr.const]),
> - an *init-declarator* (9.3 [dcl.decl]) **, an *initializer* of a structured binding declaration (9.1 [dcl.pre]),** or a *mem-initializer* (11.9.3 [class.base.init]), including the constituent expressions of the initializer,
> - an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (6.7.7 [class.temporary]) whose lifetime has not been extended, or
> - an expression that is not a subexpression of another expression and that is not otherwise part of a full-expression.
>
> ...

2. Change in 9.6 [dcl.struct.bind] paragraph 4 as follows:

> ... Each $v_i$ is the name of an lvalue of type $T_i$ that refers to the object bound to $r_i$; the referenced type is $T_i$. **The initialization of e is sequenced before the initialization of any $r_i$. The initialization of each $r_i$ is sequenced before the initialization of any $r_j$ where i < j.**

3. Append a new paragraph at the end of 9.6 [dcl.struct.bind] as follows:

> ... [ Example: ... The type of the id-expression x is "int", the type of the id-expression y is "const volatile double". -- end example ]

The initialization of *e* and of any $r_i$ are sequenced before the destruction of any temporary object introduced by the *initializer* for *e* or by the *initializer*s for the $r_i$. The temporary objects are destroyed in the reverse order of their construction.

**CWG 2024-06-14**

The specification for the lifetime of temporaries should be moved to 6.7.7 [class.temporary].)

**Proposed resolution (approved by CWG 2024-06-28):**

1. Change in 6.7.7 [class.temporary] paragraph 5 as follows:

   There are ~~four~~ **five** contexts in which temporaries are destroyed at a different point than the end of the full-expression. ...

2. Insert a new paragraph after 6.7.7 [class.temporary] paragraph 7 as follows:

   The fourth context is when a temporary object is created in the *for-range-initializer* of a range-based for statement. If such a temporary object would otherwise be destroyed at the end of the *for-range-initializer* full-expression, the object persists for the lifetime of the reference initialized by the *for-range-initializer*.

   The fifth context is when a temporary object is created in a structured binding declaration (9.6 [dcl.struct.bind]). Any temporary objects introduced by the *initializer*s for the variables with unique names are destroyed at the end of the structured binding declaration.

3. Change in 6.9.1 [intro.execution] paragraph 5 as follows:

   A *full-expression* is
   - an unevaluated operand (7.2.3 [expr.context]),
   - a *constant-expression* (7.7 [expr.const]),
   - an immediate invocation (7.7 [expr.const]),
   - an *init-declarator* (9.3 [dcl.decl]) **(including such introduced by a structured binding (9.6 [dcl.struct.bind]))** or a *mem-initializer* (11.9.3 [class.base.init]), including the constituent expressions of the initializer,
   - an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (6.7.7 [class.temporary]) whose lifetime has not been extended, or
   - an expression that is not a subexpression of another expression and that is not otherwise part of a full-expression.

   ...

4. Change in 9.6 [dcl.struct.bind] paragraph 4 as follows:

   ... Each $v_i$ is the name of an lvalue of type $T_i$ that refers to the object bound to $r_i$; the referenced type is $T_i$. **The initialization of e is sequenced before the initialization of any $r_i$. The initialization of each $r_i$ is sequenced before the initialization of any $r_j$ where i < j.**

---

**2869. `this` in local classes**

**Section:** 7.5.2 [expr.prim.this] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-03-14

(From submission #515.)

Consider:

```
struct A {
  static void f() {
    struct B {
      void *g() { return this; }
    };
  }
};
```

According to 7.5.2 [expr.prim.this] paragraph 3, this example is ill-formed, because `this` "appears within" the declaration of a static member function. The qualification "of the current class" can be read as attaching to explicit object member functions only.

Suggested resolution [SUPERSEDED]:

Change in 7.5.2 [expr.prim.this] paragraph 3 as follows:

> If a declaration declares a member function or member function template of a class X, the expression `this` is a prvalue of type "pointer to *cv-qualifier-seq* X" wherever X is the current class between the optional *cv-qualifier-seq* and the end of the *function-definition*, *member-declarator* , or *declarator*. ~~It shall not appear within the~~ **The** declaration ~~of either~~ **that determines the type of `this` shall declare neither** a static member function ~~or~~ **nor** an explicit object member function of the current class (although its type and value category are defined within such member functions as they are within an implicit object member function).

**CWG 2024-05-03**

CWG preferred a smaller surgery to avoid the English parsing issue.

**Proposed resolution (approved by CWG 2024-05-17):**

Change in 7.5.2 [expr.prim.this] paragraph 3 as follows:

> If a declaration declares a member function or member function template of a class X, the expression `this` is a prvalue of type "pointer to *cv-qualifier-seq* X" wherever X is the current class between the optional *cv-qualifier-seq* and the end of the *function-definition*, *member-declarator*, or *declarator*. It shall not appear within the declaration of ~~either~~ a static ~~member function~~ or ~~an~~ explicit object member function of the current class (although its type and value category are defined within such member functions as they are within an implicit object member function).

---

### 2870. Combining absent *encoding-prefix*es

**Section:** 5.13.5 [lex.string] **Status:** ready **Submitter:** Jan Schultke **Date:** 2024-03-09

(From submission #511.)

Subclause 5.13.5 [lex.string] paragraph 7 does not properly handle the case where both *encoding-prefix*es are absent:

> The common *encoding-prefix* for a sequence of adjacent *string-literal*s is determined pairwise as follows: If two *string-literal*s have the same *encoding-prefix* , the common *encoding-prefix* is that *encoding-prefix* . If one *string-literal* has no *encoding-prefix* , the common *encoding-prefix* is that of the other *string-literal*. Any other combinations are ill-formed.

Possible resolution [SUPERSEDED]:

Change in 5.13.5 [lex.string] paragraph 7 as follows:

> The common *encoding-prefix* for a sequence of adjacent *string-literal*s is determined pairwise as follows: ~~If two *string-literal*s have the same *encoding-prefix*, the common *encoding-prefix* is that *encoding-prefix*.~~ If one *string-literal* has no *encoding-prefix*, the common *encoding-prefix* is that of the other *string-literal* **or is absent if the other *string-literal* has no *encoding-prefix*. Otherwise, both *string-literal*s shall have the same *encoding-prefix*, and the common *encoding-prefix* is that *encoding-prefix***. ~~Any other combinations are ill-formed.~~

**CWG 2024-05-03**

CWG preferred a holistic instead of a pairwise approach.

**Proposed resolution (approved by CWG 2024-05-17):**

Change in 5.13.5 [lex.string] paragraph 7 as follows:

> ~~The common *encoding-prefix* for a sequence of adjacent *string-literal*s is determined pairwise as follows: If two *string-literal*s have the same *encoding-prefix* , the common *encoding-prefix* is that *encoding-prefix* . If one *string-literal* has no *encoding-prefix* , the common *encoding-prefix* is that of the other *string-literal*. Any other combinations are ill-formed.~~
>
> The *string-literal*s in any sequence of adjacent *string-literal*s shall have at most one unique *encoding-prefix* among them. The common *encoding-prefix* of the sequence is that *encoding-prefix*, if any.

---

### 2871. User-declared constructor templates inhibiting default constructors

**Section:** 11.4.5.2 [class.default.ctor] **Status:** ready **Submitter:** Jim X **Date:** 2024-03-03

(From submission #510.)

Subclause 11.4.5.2 [class.default.ctor] paragraph 1 does not, but should, consider user-declared constructor templates.

**Proposed resolution (approved by CWG 2024-04-05):**

Change in 11.4.5.2 [class.default.ctor] paragraph 1 as follows:

> ... If there is no user-declared constructor **or constructor template** for class X, a non-explicit constructor having no parameters is implicitly declared as defaulted (9.5 [dcl.fct.def]). ...

---

### 2872. Linkage and unclear "can be referred to"

**Section:** 6.6 [basic.link] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-03-08

It is unclear what "can be referred to" means in 6.6 [basic.link] paragraph 2. Paragraph 8 provides precise definitions for "same entity".

Possible resolution [SUPERSEDED]:

1. Replace in 6.6 [basic.link] paragraph 2 as follows:

> ~~A name is said to have *linkage* when it can denote the same object, reference, function, type, template, namespace or value as a name introduced by a declaration in another scope:~~
> - ~~When a name has external linkage, the entity it denotes can be referred to by names from scopes of other translation units or from other scopes of the same translation unit.~~
> - ~~When a name has module linkage, the entity it denotes can be referred to by names from other scopes of the same module unit (10.1 [module.unit]) or from scopes of other module units of that same module.~~
> - ~~When a name has internal linkage, the entity it denotes can be referred to by names from other scopes in the same translation unit.~~
> - ~~When a name has no linkage, the entity it denotes cannot be referred to by names from other scopes.~~
>
> A name can have *external linkage*, *module linkage*, *internal linkage*, or *no linkage*, as determined by the rules below. [ Note: The linkage of a name determines when corresponding declarations (6.4.1 [basic.scope.scope]) that introduce that name can declare the same entity. ---end note]

2. Change in 11.6 [class.local] paragraph 3 as follows:

> ~~If class X is a local class, a nested class Y may be declared in class X and later defined in the definition of class X or be later defined in the same scope as the definition of class X.~~ A class nested within a local class is a local class. **A member of a local class X shall be declared only in the definition of X or the nearest enclosing block scope of X.**

**CWG 2024-05-03**

CWG opined to split off the second change (see issue 2890) and clarify the note.

**Proposed resolution (approved by CWG 2024-05-17):**

Replace in 6.6 [basic.link] paragraph 2 as follows:

> ~~A name is said to have *linkage* when it can denote the same object, reference, function, type, template, namespace or value as a name introduced by a declaration in another scope:~~
>
> - ~~When a name has external linkage, the entity it denotes can be referred to by names from scopes of other translation units or from other scopes of the same translation unit.~~
> - ~~When a name has module linkage, the entity it denotes can be referred to by names from other scopes of the same module unit (10.1 [module.unit]) or from scopes of other module units of that same module.~~
> - ~~When a name has internal linkage, the entity it denotes can be referred to by names from other scopes in the same translation unit.~~
> - ~~When a name has no linkage, the entity it denotes cannot be referred to by names from other scopes.~~
>
> A name can have *external linkage*, *module linkage*, *internal linkage*, or *no linkage*, as determined by the rules below. [ Note: All declarations of an entity with a name with internal linkage appear in the same translation unit. All declarations of an entity with module linkage are attached to the same module. ---end note]

### 2874. Qualified declarations of partial specializations

**Section:** 9.2.9.5 [dcl.type.elab] **Status:** ready **Submitter:** Krystian Stasiowski **Date:** 2024-03-19

(From submission #521.)

Consider:

```
namespace N {
  template<typename T>
  struct A;
}

template<>
struct N::A<int>;      // OK

template<typename T>
struct N::A<T*>;       // error: invalid use of elaborated-type-specifier with a qualified-id
```

However, all major implementations support this.

**Proposed resolution (approved by CWG 2024-05-17):**

Change in 9.2.9.5 [dcl.type.elab] paragraph 2 as follows:

> If an *elaborated-type-specifier* is the sole constituent of a declaration, the declaration is ill-formed unless it is an explicit specialization (13.9.4 [temp.expl.spec]), **a partial specialization (13.7.6 [temp.spec.partial]),** an explicit instantiation (13.9.3 [temp.explicit])**,** or it has one of the following forms:
>
> ```
> class-key attribute-specifier-seqopt identifier ;
> class-key attribute-specifier-seqopt simple-template-id ;
> ```
>
> In the first case, the *elaborated-type-specifier* declares the identifier as a *class-name*. The second case shall appear only in an *explicit-specialization* (13.9.4 [temp.expl.spec]) or in a *template-declaration* (where it declares a partial specialization ~~(13.7 [temp.decls])~~. The *attribute-specifier-seq*, if any, appertains to the class or template being declared.

---

### 2876. Disambiguation of `T x = delete("text")`

**Section:** 9.5.1 [dcl.fct.def.general] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-03-22

P2573R2 (= delete("should have a reason");), adopted in Tokyo, does not disambiguate the following syntax:

```
using T = void ();
using U = int;

T a = delete ("hello");
U b = delete ("hello");
```

Either may be parsed as a (semantically ill-formed) *simple-declaration* whose initializer is a *delete-expression* or as a *function-definition*.

**Proposed resolution (approved by CWG 2024-05-31):**

Change and split 9.1 [dcl.pre] paragraph 9 as follows:

An object definition causes storage of appropriate size and alignment to be reserved and any appropriate initialization (9.4 [dcl.init]) to be done.

Syntactic components beyond those found in the general form of *simple-declaration* are added to a function declaration to make a *function-definition*. **A token sequence starting with { or = is treated as a *function-body* (9.5.1 [dcl.fct.def.general]) if the type of the *declarator-id* (9.3.4.1 [dcl.meaning.general]) is a function type, and is otherwise treated as a *brace-or-equal-initializer* (9.4.1 [dcl.init.general]). [ Note: If the declaration acquires a function type through template instantiation, the program is ill-formed; see 13.9.1 [temp.spec.general]. The function type of a function definition cannot be specified with a *typedef-name* (9.3.4.6 [dcl.fct]). --end note ]** ~~An object declaration, however, is also a definition unless it contains the extern specifier and has no initializer (6.2 [basic.def]). An object definition causes storage of appropriate size and alignment to be reserved and any appropriate initialization (9.4 [dcl.init]) to be done.~~

This drafting also resolves issue 2144.

---

### 2877. Type-only lookup for *using-enum-declarator*

**Section:** 9.7.2 [enum.udecl] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-04-07

Issue 2621 claimed to ask the question whether lookup for `using enum` declarations was supposed to be type-only, but the example actually highlighted the difference between *elaborated-type-specifier* lookup (where finding nothing but `typedef` names is ill-formed) and ordinary lookup.

However, consider:

```
enum A {
  x, y
};
void f() {
  int A;
  using enum A;       // #1, error: names non-type int A
  using T = enum A;   // #2, OK, names ::A
}
```

The two situations should both be type-only lookups for consistency, although #2 would not find `typedefs`.

**Proposed resolution (reviewed by CWG 2024-05-17) [SUPERSEDED]:**

Change in 9.7.2 [enum.udecl] paragraph 1 as follows:

A *using-enum-declarator* names the set of declarations found by **type-only** lookup (**6.5.1 [basic.lookup.general]** ~~6.5.3 [basic.lookup.unqual], 6.5.5 [basic.lookup.qual]~~) for the *using-enum-declarator* **(6.5.3 [basic.lookup.unqual], 6.5.5 [basic.lookup.qual])**. The *using-enum-declarator* shall designate a non-dependent type with a reachable *enum-specifier*.

**Additional notes (May, 2024)**

An example is desirable. Also, the treatment of the following example is unclear:

```
template<class T> using AA = T;
enum AA<E> e; // ill-formed elaborated-type-specifier
using enum AA<E>; // Clang and MSVC reject, GCC and EDG accept
```

**Proposed resolution (approved by CWG 2024-06-26):**

Change in 9.7.2 [enum.udecl] paragraph 1 as follows:

> A *using-enum-declarator* names the set of declarations found by **type-only** lookup (**6.5.1 [basic.lookup.general]** ~~6.5.3 [basic.lookup.unqual], 6.5.5 [basic.lookup.qual]~~) for the *using-enum-declarator* **(6.5.3 [basic.lookup.unqual], 6.5.5 [basic.lookup.qual])**. The *using-enum-declarator* shall designate a non-dependent type with a reachable *enum-specifier*. **[ Example:**
>
> ```
> enum E { x };
> void f() {
>   int E;
>   using enum E;   // OK
> }
> using F = E;
> using enum F;     // OK
> template<class T> using EE = T;
> void g() {
>   using enum EE<E>;  // OK
> }
> ```
>
> **-- end example ]**

---

**2881. Type restrictions for the explicit object parameter of a lambda**

**Section:** 7.5.5.2 [expr.prim.lambda.closure] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-04-19

Subclause 7.5.5.2 [expr.prim.lambda.closure] paragraph 5 restricts the type of an explicit object parameter of a lambda to the closure type or classes derived from the closure type. It neglects to consider ambiguous or private derivation scenarios.

**Proposed resolution (approved by CWG 2024-06-26):**

1. Change in 7.5.5.2 [expr.prim.lambda.closure] paragraph 5 as follows:

   > Given a lambda with a *lambda-capture*, the type of the explicit object parameter, if any, of the lambda's function call operator (possibly instantiated from a function call operator template) shall be either:
   > - the closure type
   > - a class type **publicly and unambiguously** derived from the closure type, or
   > - a reference to a possibly cv-qualified such type.

2. Add a new bullet after 13.10.3.1 [temp.deduct.general] bullet 11.10:

   > - ...
   > - Attempting to create a function type in which a parameter has a type of void, or in which the return type is a function type or array type.
   > - Attempting to give to an explicit object parameter of a lambda's function call operator a type not permitted for such (7.5.5.2 [expr.prim.lambda.closure]).

**CWG 2024-06-26**

The following example is not supported by the proposed resolution and remains ill-formed:

```
int main() {
  int x = 0;
  auto lambda = [x] (this auto self) { return x; };
  using Lambda = decltype(lambda);
  struct D : private Lambda {
    D(Lambda l) : Lambda(l) {}
    using Lambda::operator();
    friend Lambda;
  } d(lambda);
  d();
}
```

---

**2882. Unclear treatment of conversion to `void`**

**Section:** 7.6.1.9 [expr.static.cast] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-04-24

The ordering of paragraphs in 7.6.1.9 [expr.static.cast] appears to imply that an attempt is made to form an implicit conversion sequence for conversions to `void`, possibly considering user-defined conversion functions to `void`. This is not intended.

**Proposed resolution (approved by CWG 2024-05-31):**

1. Move 7.6.1.9 [expr.static.cast] paragraph 6 to before paragraph 4, strike paragraph 5, and adjust paragraph 7:

> Any expression can be explicitly converted to type *cv* `void`, in which case the operand is a discarded-value expression (7.2 [expr.prop]). [*Note 3:* Such a static_cast has no result as it is a prvalue of type void; see 7.2.1 [basic.lval]. —*end note*] [*Note 4:* However, if the value is in a temporary object (6.7.7 [class.temporary]), the destructor for that object is not executed until the usual time, and the value of the object is preserved for the purpose of executing the destructor. —*end note*]
>
> ~~An~~ **Otherwise, an** expression E can be explicitly converted to a type T if there is an implicit conversion sequence (12.2.4.2 [over.best.ics]) from E to T, if overload resolution ...
>
> ~~Otherwise, the static_cast shall perform one of the conversions listed below. No other conversion shall be performed explicitly using a static_cast.~~
>
> ~~Any expression can be explicitly converted to type cv void, in which case the operand is a discarded-value expression (7.2 [expr.prop]). [*Note 3:* Such a static_cast has no result as it is a prvalue of type void; see 7.2.1 [basic.lval]. —*end note*] [*Note 4:* However, if the value is in a temporary object (6.7.7 [class.temporary]), the destructor for that object is not executed until the usual time, and the value of the object is preserved for the purpose of executing the destructor. —*end note*]~~
>
> ~~The~~ **Otherwise, the** inverse of ~~any~~ **a** standard conversion sequence (7.3 [conv]) not containing an lvalue-to-rvalue (7.3.2 [conv.lval]), array-to-pointer (7.3.3 [conv.array]), function-to-pointer (7.3.4 [conv.func]), null pointer (7.3.12 [conv.ptr]), null member pointer (7.3.13 [conv.mem]), boolean (7.3.15 [conv.bool]), or function pointer (7.3.14 [conv.fctptr]) conversion, can be performed explicitly using static_cast. A program is ill-formed if it uses static_cast to perform the inverse of an ill-formed standard conversion sequence.

2. Add a new paragraph after 7.6.1.9 [expr.static.cast] paragraph 14:

> A prvalue of type “pointer to cv1 void” can be converted to a prvalue of type “pointer to cv2 T”, ... Otherwise, the pointer value is unchanged by the conversion. [*Example 3:* ... —*end example*]

No other conversion can be performed using `static_cast`.

---

**2883. Definition of "odr-usable" ignores lambda scopes**

**Section:** 6.3 [basic.def.odr] **Status:** ready **Submitter:** Hubert Tong **Date:** 2024-03-20

(From submission #523.)

Consider:

```
void f() {
  int x;
  [&] {
    return x;       // #1
  };
}
```

The odr-use of `x` is ill-formed, because `x` is not odr-usable in that scope, because there is a lambda scope (6.4.5 [basic.scope.lambda]) between #1 and the definition of `x`.

A more involved example exhibits implementation divergence:

```
struct A {
  A() = default;
  A(const A &) = delete;
  constexpr operator int() { return 42; }
};
void f() {
  constexpr A a;
  [=]<typename T, int = a> {};    // OK, not odr-usable from a default template argument, and not odr-used
}
```

**Proposed resolution (approved by CWG 2024-05-31):**

Change in 6.3 [basic.def.odr] paragraph 10 as follows:

> A local entity (6.1 [basic.pre]) is *odr-usable* in a scope (6.4.1 [basic.scope.scope]) if:
>
> - either the local entity is not *this, or an enclosing class or non-lambda function parameter scope exists and, if the innermost such scope is a function parameter scope, it corresponds to a non-static member function, and
> - for each intervening scope (6.4.1 [basic.scope.scope]) between the point at which the entity is introduced and the scope (where *this is considered to be introduced within the innermost enclosing class or non-lambda function definition scope), either:
>   - the intervening scope is a block scope, or
>   - the intervening scope is the function parameter scope of a *lambda-expression*, **or**
>   - **the intervening scope is the lambda scope of a *lambda-expression*** that has a *simple-capture* naming the entity or has a *capture-default*, and the block scope of the *lambda-expression* is also an intervening scope.
>
> If a local entity is odr-used in a scope in which it is not odr-usable, the program is ill-formed.

---

**2886. Temporaries and trivial potentially-throwing special member functions**

**Section:** 6.7.7 [class.temporary] **Status:** ready **Submitter:** Jiang An **Date:** 2024-04-27

(From submission #531.)

Paper P1286R2 (Contra CWG DR1778) allowed trivial potentially-throwing special member functions. Whether such a trivial special member function is actually invoked is thus observable via the `noexcept` operator. Issue 2820 clarified the situation for value-initialization and removed a special case for a trivial default constructor.

Subclause 6.7.7 [class.temporary] paragraph 4 appears to normatively avoid invoking a trivial constructor or destructor, something best left to the as-if rule. There is implementation divergence for this example:

```
struct C {
  C() = default;
  ~C() noexcept(false) = default;
};

static_assert(noexcept(C()));
```

A related question arises from the introduction of temporaries for function parameters and return values in 6.7.7 [class.temporary] paragraph 3. However, this situation can never result in actually throwing an exception during forward evolution: when the constructor becomes non-trivial, the permission to create a temporary object evaporates.

**Proposed resolution (approved by CWG 2024-05-31):**

1. Change in 6.7.7 [class.temporary] paragraph 3 as follows:

> When an object of class type X is passed to or returned from a **potentially-evaluated** function **call**, if X has at least one eligible copy or move constructor (11.4.4 [special]), each such constructor is trivial, and the destructor of X is either trivial or deleted, implementations are permitted to create a temporary object to hold the function parameter or result object. The temporary object is constructed from the function argument or return value, respectively, and the function's parameter or return object is initialized as if by using the eligible trivial constructor to copy the temporary (even if that constructor is inaccessible or would not be selected by overload resolution to perform a copy or move of the object).

2. Change in 6.7.7 [class.temporary] paragraph 4 as follows:

> ~~When an implementation introduces a temporary object of a class that has a non-trivial constructor (11.4.5.2 [class.default.ctor], 11.4.5.3 [class.copy.ctor]), it shall ensure that a constructor is called for the temporary object. Similarly, the destructor shall be called for a temporary with a non-trivial destructor (11.4.7 [class.dtor]).~~ Temporary objects are destroyed as the last step in evaluating the full-expression (6.9.1 [intro.execution]) that (lexically) contains the point where they were created. This is true even if that evaluation ends in throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full-expression, not with any specific subexpression.

---

**2887. Missing compatibility entries for xvalues**

**Section:** C.6.3 [diff.cpp03.expr] **Status:** ready **Submitter:** Jiang An **Date:** 2024-04-18

(From submission #528.)

Paper N3055 (A Taxonomy of Expression Value Categories) introduced xvalues. This changed the behavior of well-formed code for `typeid` and the conditional operator, but corresponding entries in Annex C are missing.

**Proposed resolution (approved by CWG 2024-05-31):**

1. Add a new paragraph to C.6.3 [diff.cpp03.expr] as follows:

> **Affected subclause:** 7.6.1.8 [expr.typeid]
> **Change:** Evaluation of operands in `typeid`.
> **Rationale:** Introduce additional expression value categories.
> **Effect on original feature:** Valid C++ 2003 code that uses xvalues as operands for `typeid` may change behavior. For example,
>
> ```
> void f() {
>   struct B {
>     B() {}
>     virtual ~B() { }
>   };
>
>   struct C { B b; };
>   typeid(C().b); // unevaluated in C++03, evaluated in C++11
> }
> ```

2. Add a new paragraph to C.6.3 [diff.cpp03.expr] as follows:

> **Affected subclause:** 7.6.16 [expr.cond]
> **Change:** Fewer copies in the conditional operator.
> **Rationale:** Introduce additional expression value categories.
> **Effect on original feature:** Valid C++ 2003 code that uses xvalues as operands for the conditional operator may change behavior. For example,
>
> ```
> void f() {
>   struct B {
>     B() {}
>     B(const B&) { }
>   };
>   struct D : B {};
>
>   struct BB { B b; };
>   struct DD { D d; };
>
>   true ? BB().b : DD().d; // additional copy in C++03, no copy or move in C++11
> }
> ```

---

### 2891. Normative status of implementation limits

**Section:** Clause Annex B [implimits] **Status:** ready **Submitter:** ISO/CS **Date:** 2024-05-17

It is unclear whether Clause Annex B [implimits] is normative, and what its normative content is. The Annex was editorially switched from "informative" to "normative" in September 2020 with this change description:

> [intro.compliance.general, implimits] Cite Annex B normatively.
>
> This change also promotes Annex B [implimits] to a "normative" annex. The existing wording in the annex is already normative in character.

On the other hand, this sentence in Clause Annex B [implimits] paragraph 2 seems to say otherwise:

> ... However, these quantities are only guidelines and do not determine compliance.

If it is indeed intentional that the actual quantitative limits are just guidelines, then the normative (conformance-related) content is limited to Clause Annex B [implimits] paragraph 1, which establishes documentation requirements only:

> Because computers are finite, C++ implementations are inevitably limited in the size of the programs they can successfully process. Every implementation shall document those limitations where known. This documentation may cite fixed limits where they exist, say how to compute variable limits as a function of available resources, or say that fixed limits do not exist or are unknown.

However, those general documentation requirements are best expressed in 4.1.1 [intro.compliance.general], leaving Clause Annex B [implimits] with just an informative list of minimum suggested quantities.

Possible resolution [SUPERSEDED]:

1. Change in 4.1.1 [intro.compliance.general] bullet 2.1 as follows:

   > If a program contains no violations of the rules in Clause 5 through Clause 33 and Annex D, a conforming implementation shall, within its resource limits ~~as described in Annex B~~ **(see below)**, accept and correctly execute that program.

2. Insert a new paragraph before 4.1.1 [intro.compliance.general] paragraph 8 as follows:

   > An implementation shall document its limitations in the size of the programs it can successfully process, where known. This documentation may cite fixed limits where they exist, say how to compute variable limits as a function of available resources, or say that fixed limits do not exist or are unknown. Clause Annex B [implimits] lists some quantities that can be subject to limitations, and recommends a minimum value for each quantity.
   >
   > A conforming implementation may have extensions (including additional library functions), ...

3. Change in Clause Annex B [implimits] paragraph 1 and paragraph 2 as follows:

   > Annex B
   > (~~normative~~**informative**)
   >
   > ~~Because computers are finite, C++ implementations are inevitably limited in the size of the programs they can successfully process. Every implementation shall document those limitations where known. This documentation may cite fixed limits where they exist, say how to compute variable limits as a function of available resources, or say that fixed limits do not exist or are unknown.~~
   >
   > ~~The limits may constrain quantities that include those described below or others.~~ **Implementations can exhibit limitations, among others for the quantities in the following list.** The bracketed number following each quantity is recommended as the minimum **value** for that quantity. ~~However, these quantities are only guidelines and do not determine compliance.~~

**CWG 2024-05-17**

CWG was divided whether to relax the documentation requirement or not. As a next step, the wording for a relaxation should be reviewed.

**Proposed resolution (approved by CWG 2024-06-26):**

1. Change in 4.1.1 [intro.compliance.general] bullet 2.1 as follows:

> If a program contains no violations of the rules in Clause 5 through Clause 33 and Annex D, a conforming implementation shall~~, within its resource limits as described in Annex B,~~ accept and correctly execute that program**, except when the implementation's limitations (see below) are exceeded**.

2. Insert a new paragraph before 4.1.1 [intro.compliance.general] paragraph 8 as follows:

> An implementation is encouraged to document its limitations in the size or complexity of the programs it can successfully process, if possible and where known. Clause Annex B [implimits] lists some quantities that can be subject to limitations and a potential minimum supported value for each quantity.
>
> A conforming implementation may have extensions (including additional library functions), ...

3. Change in Clause Annex B [implimits] paragraph 1 and paragraph 2 as follows:

> Annex B
> (~~normative~~**informative**)
>
> ~~Because computers are finite, C++ implementations are inevitably limited in the size of the programs they can successfully process. Every implementation shall document those limitations where known. This documentation may cite fixed limits where they exist, say how to compute variable limits as a function of available resources, or say that fixed limits do not exist or are unknown.~~
>
> ~~The limits may constrain quantities that include those described below or others.~~ **Implementations can exhibit limitations for various quantities; some possibilities are presented in the following list.** The bracketed number following each quantity is ~~recommended as the~~ **a potential** minimum **value** for that quantity. ~~However, these quantities are only guidelines and do not determine compliance.~~

---

**2892. Unclear usual arithmetic conversions**

**Section:** 7.4 [expr.arith.conv] **Status:** ready **Submitter:** Lauri Vasama **Date:** 2024-05-06

(From submission #533.)

Subclause 7.4 [expr.arith.conv] bullet 1.4.1 specifies:

- If both operands have the same type, no further conversion is needed.

What does "needed" mean?

**Proposed resolution (approved by CWG 2024-05-31):**

Change in 7.4 [expr.arith.conv] bullet 1.4.1 as follows:

- If both operands have the same type, no further conversion is ~~needed~~ **performed**.

---

**2895. Initialization should ignore the destination type's cv-qualification**

**Section:** 9.4.1 [dcl.init.general] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-05-29

(From submission #541.)

Initialization of *cv* `T` follows the same rules as initialization of `T`. Issue 2830 clarified this for list-initialization. Further amendments are needed to properly handle cv-qualified versions of `char8_t` and `char16_t` as well as `bool`.

**Proposed resolution (approved by CWG 2024-06-14):**

1. Change in 9.4.1 [dcl.init.general] paragraph 16 as follows:

> The semantics of initializers are as follows. The destination type is the **cv-unqualified** type of the object or reference being initialized and the source type is the type of the initializer expression. If the initializer is not a single (possibly parenthesized) expression, the source type is not defined.

2. Change in 9.4.1 [dcl.init.general] bullet 16.6 as follows:

- Otherwise, if the destination type is a ~~(possibly cv-qualified)~~ class type:
  - If the initializer expression is a prvalue and the cv-unqualified version of the source type is the same ~~class as the class of~~ **as** the destination **type**, the initializer expression is used to initialize the destination object. [*Example 2:* T x = T(T(T())); value-initializes x. *—end example*]
  - Otherwise, if the initialization is direct-initialization, or if it is copy-initialization where the cv-unqualified version of the source type is the same ~~class as, or a derived class of, the class of~~ **as or is derived from** the destination **type**, constructors are considered. The applicable constructors are enumerated (12.2.2.4 [over.match.ctor]), and the best one is chosen through overload resolution (12.2 [over.match]). Then:

3. Change in 9.4.1 [dcl.init.general] bullet 16.9 as follows:

- Otherwise, the initial value of the object being initialized is the (possibly converted) value of the initializer expression. A standard conversion sequence (7.3 [conv]) is used to convert the initializer expression to a prvalue of ~~the cv-unqualified version of~~ the destination type; no user-defined conversions are considered. If the conversion cannot be done, the initialization is ill-formed. When initializing a bit-field with a value that it cannot represent, the resulting value of the bit-field is implementation-defined. [ Note: ... ]